

Module 22: What's New, Honestly

This module is where the curriculum is honest about its own title.

Module 01 claimed that JDK 27 is the runtime and toolchain, while the language you learn is the stable Java accumulated through JDK 25. Twenty-one modules later, here is the evidence.

Every version claim below links to its source. That habit is the real subject of this module, because everything else in it will go out of date.

What You'll Learn

- What JDK 26 and 27 actually delivered, from the release pages
- The six-month cadence, LTS, and which number matters for what
- How to read a JEP, and what the preview round number tells you
- Structured concurrency and primitive patterns, run behind `--enable-preview`

Coming From Another Language

	Python	Node.js	Java
Release cadence	~annual	~biannual major	every six months, fixed date
Long-term support	~5 years per minor	even-numbered majors, 30 months	LTS every two years
Trying unfinished features	<code>from __future__ import</code>	flags, or a release channel	<code>--enable-preview</code>
Where the spec lives	PEPs	TC39 proposals	JEPs

The closest analogue is TC39's staged proposals. A Java preview is roughly a stage 3 proposal: specified, implemented, shipped behind a flag, and still able to change before it lands.

The difference worth knowing: a Python `__future__` import is forward compatible, while a Java **preview class file is rejected outright** by a different release, even with the flag. That single fact decides most of what this module recommends.

The Lesson

What actually shipped

JDK 26, GA 17 March 2026:

JEP	Title	Status
500	Prepare to Make Final Mean Final	Final
504	Remove the Applet API	Final
516	Ahead-of-Time Object Caching with Any GC	Final
517	HTTP/3 for the HTTP Client API	Final
522	G1 GC: Improve Throughput by Reducing Synchronization	Final
524	PEM Encodings of Cryptographic Objects	Second Preview
525	Structured Concurrency	Sixth Preview
526	Lazy Constants	Second Preview
529	Vector API	Eleventh Incubator
530	Primitive Types in Patterns	Fourth Preview

JDK 27, GA 15 September 2026:

JEP	Title	Status
523	Make G1 the Default Garbage Collector in All Environments	Final
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	Final
531	Lazy Constants	Third Preview
532	Primitive Types in Patterns	Fifth Preview
533	Structured Concurrency	Seventh Preview
534	Compact Object Headers by Default	Final
536	JFR In-Process Data Redaction	Final
537	Vector API	Twelfth Incubator
538	PEM Encodings of Cryptographic Objects	Third Preview

Nothing in JDK 27 changes how you write Java. Four of its five finalised JEPs are runtime and security work: a collector default, smaller object headers, TLS key exchange, flight recorder redaction. You benefit from all four without editing a line. Everything language-shaped is still in preview, some of it for the seventh consecutive release.

A course promising "the new features of Java 27" would have had to pad the list or teach preview features as settled. Both are worse than saying so.

The two in JDK 26 worth knowing about: [JEP 517](#) gives the built-in `HttpClient` HTTP/3, a real capability you get by changing a version request rather than adding a dependency. [JEP 500](#) warns when

code mutates a `final` field by reflection, ahead of forbidding it, which is aimed squarely at some serialisation and mocking libraries.

Releases and LTS

A feature release every six months, in March and September, on a fixed date whether or not anything is ready. Before 2017, releases were feature-driven and arrived every two to three years.

Release	Designation	Date
Java 17	LTS	September 2021
Java 21	LTS	September 2023
Java 25	LTS	September 2025
Java 26		March 2026
Java 27		September 2026

A non-LTS gets security patches until the next release supersedes it. Six months, then nothing. An LTS gets years, which is why nearly every production system runs one.

"What is new in Java 27" is the wrong question for most teams. The right one is "what is new since Java 21", which is where they actually are.

Situation	Use
Learning	the newest release. Features finalise here first.
Production	the current LTS, today Java 25
A library	the oldest LTS you must support

The accumulation nobody notices. Any single release looks thin. Between Java 21 and Java 25, all of these arrived, and every one appears in this curriculum:

JEP	Feature	Module
512	Compact source files and instance main methods	01
511	Module import declarations	20
513	Flexible constructor bodies	05
506	Scoped values	19
485	Stream gatherers	14
491	Synchronize virtual threads without pinning	19

Use `Runtime.version()` to check what you are on, not the `java.version` string. Parsing that string broke for everyone when Java 9 changed its format from `1.8.0_301` to `9.0.1`.

How to read a JEP

Anything you read about Java goes out of date, including this file. JEPs do not, because they **are** the specification.

- openjdk.org/jeps/0 is the index
- `openjdk.org/jeps/<number>` is one JEP
- `openjdk.org/projects/jdk/<version>` is what shipped in a release

The fields, in order of usefulness:

Field	Why
Status	only <i>Closed/Delivered</i> means it shipped
Release	no Release field means it is not scheduled for anything
Summary	one paragraph, often all you need
Motivation	<i>why</i> it exists. The section people skip and the one worth reading.
Alternatives	what was rejected. Answers most "why didn't they just..." questions.

The vocabulary:

Term	Meaning
Preview	complete and specified, not yet permanent. Needs <code>--enable-preview</code> . May change or be withdrawn.
Incubator	an API in a <code>jdk.incubator</code> module. Needs <code>--add-modules</code> . Less settled than preview.
Experimental	a JVM feature behind <code>-XX:+UnlockExperimentalVMOptions</code>
Final	permanent. It will not be removed.

The preview round number is a practical signal, not trivia. Structured concurrency is on its seventh. Pattern matching for switch took four. Records took two. A high number means the design is still being argued about and the API has usually changed between rounds, which is exactly what happened here: code written against the earlier structured concurrency previews does not compile against this one.

Three questions to ask about any feature:

1. Is it Final? If not, what round, and how many came before?
2. Which release delivered it, and is that an LTS?
3. Can the minimum version you must support actually use it?

The preview features, run

Both examples below carry an `// EXPECT: preview` marker, so the verification script runs them with `-enable-preview --source 27`.

Structured concurrency, seventh preview. Module 18's `ExecutorService` lets you submit two tasks and forget one. Nothing connects them, and a `Future` nobody inspects swallows its exception.

```
try (var scope = StructuredTaskScope.open()) {
    var user = scope.fork(() -> loadUser());
    var orders = scope.fork(() -> loadOrders());
    scope.join();
    return combine(user.get(), orders.get());
}
```

```
user:ada and orders:3
```

```
scope failed: ExecutionException
cause: the fast one failed
```

The second case returns immediately. A two-second sibling was **cancelled** the moment the fast task threw, rather than being waited on pointlessly.

And it closes a loop from Module 19:

```
child sees: req-7
```

Module 19 said a plain child thread does not inherit a `ScopedValue`, and that inheritance needs structured concurrency. That is the line above. It works because the scope guarantees the parent outlives its children, so the binding cannot expire while a child is reading it.

Primitive patterns, fifth preview.

```
42 instanceof byte: true
300 instanceof byte: false
```

For a reference type, `instanceof` asks "is this object of that type". For a primitive it asks something different: **can this value convert without loss**.

Today that narrowing is a cast, and `(byte) 300` gives `44` with no warning at all. That is Module 02's overflow. The pattern form refuses rather than wrapping.

Should you use either in production? No. Preview class files are refused by a different release even with the flag, so every JDK upgrade may break your build. Read them, try them, and watch the JEP for the round where they finalise.

Run It

```
java modules/22-whats-new/examples/WhatChanged.java
java modules/22-whats-new/examples/ReleasesAndLts.java
java modules/22-whats-new/examples/HowToReadAJep.java

java --enable-preview --source 27 modules/22-whats-
    new/examples/PreviewStructuredConcurrency.java
java --enable-preview --source 27 modules/22-whats-
    new/examples/PreviewPrimitivePatterns.java

./scripts/verify-examples.sh modules/22-whats-new
```

Common Mistakes

Enabling preview features in production. A preview class file is rejected by a different JDK release even with the flag.

Chasing the newest release for a production system. Six months of patches, then nothing.

Reading a blog post instead of the JEP. The post was written against a different preview round.

Parsing `java.version`. Use `Runtime.version()`.

Assuming a preview API is stable because it works. Seven rounds means seven chances it changed.

Judging a release by its JEP count. The interesting question is what accumulated since the last LTS.

Key Takeaways

- **JDK 27 delivered nothing that changes how you write Java.** Its finalised work is runtime and security.
- **LTS is the number that matters for production;** the newest release is the one that matters for learning.
- **The value accumulates between LTS releases,** not within any single one.
- **A JEP is the specification and outlives every article about it.** Read the Motivation section.
- **The preview round number tells you how settled a design is.**
- **Never ship preview features.** The class files are rejected by other releases.

Homework

[homework/README.md](#)

Research a feature from primary sources and decide whether you could use it. Reference solution in [solutions/22-whats-new/](#).